

```
from imutils import face_utils
from utils import *
import numpy as np
import pyautogui as pag
import imutils
import dlib
import cv2

# Thresholds and consecutive frame length for triggering the mouse action.
MOUTH_AR_THRESH = 0.3
MOUTH_AR_CONSECUTIVE_FRAMES = 5
EYE_AR_THRESH = 0.20
EYE_AR_CONSECUTIVE_FRAMES = 5
WINK_AR_DIFF_THRESH = 0.001
WINK_AR_CLOSE_THRESH = 0.2
WINK_CONSECUTIVE_FRAMES = 4

# Initialize the frame counters for each action as well as
# booleans used to indicate if action is performed or not
MOUTH_COUNTER = 0
EYE_COUNTER = 0
WINK_COUNTER = 0
INPUT_MODE = False
EYE_CLICK = False
LEFT_WINK = False
RIGHT_WINK = False
SCROLL_MODE = False
ANCHOR_POINT = (0, 0)
WHITE_COLOR = (255, 255, 255)
YELLOW_COLOR = (0, 255, 255)
RED_COLOR = (0, 0, 255)
GREEN_COLOR = (0, 255, 0)
BLUE_COLOR = (255, 0, 0)
BLACK_COLOR = (0, 0, 0)

# Initialize Dlib's face detector (HOG-based) and then create
```

```
# the facial landmark predictor
shape_predictor = "model/shape_predictor_68_face_landmarks.dat"
detector = dlib.get_frontal_face_detector()
predictor = dlib.shape_predictor(shape_predictor)

# Grab the indexes of the facial landmarks for the left and
# right eye, nose and mouth respectively
(lStart, lEnd) = face_utils.FACIAL_LANDMARKS_IDXS["left_eye"]
(rStart, rEnd) = face_utils.FACIAL_LANDMARKS_IDXS["right_eye"]
(nStart, nEnd) = face_utils.FACIAL_LANDMARKS_IDXS["nose"]
(mStart, mEnd) = face_utils.FACIAL_LANDMARKS_IDXS["mouth"]

# Video capture
vid = cv2.VideoCapture(0)
resolution_w = 1366
resolution_h = 768
cam_w = 640
cam_h = 480
unit_w = resolution_w / cam_w
unit_h = resolution_h / cam_h

while True:
    # Grab the frame from the threaded video file stream, resize
    # it, and convert it to grayscale
    # channels)
    _, frame = vid.read()
    frame = cv2.flip(frame, 1)
    frame = imutils.resize(frame, width=cam_w, height=cam_h)
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Detect faces in the grayscale frame
    rects = detector(gray, 0)

    # Loop over the face detections
    if len(rects) > 0:
        rect = rects[0]
```

<

```

else:
    cv2.imshow("Frame", frame)
    key = cv2.waitKey(1) & 0xFF
    continue

# Determine the facial landmarks for the face region, then
# convert the facial landmark (x, y)-coordinates to a NumPy
# array
shape = predictor(gray, rect)
shape = face_utils.shape_to_np(shape)

# Extract the left and right eye coordinates, then use the
# coordinates to compute the eye aspect ratio for both eyes
mouth = shape[mStart:mEnd]
leftEye = shape[lStart:lEnd]
rightEye = shape[rStart:rEnd]
nose = shape[nStart:nEnd]

# Because I flipped the frame, left is right, right is left.
temp = leftEye
leftEye = rightEye
rightEye = temp

# Average the mouth aspect ratio together for both eyes
mar = mouth_aspect_ratio(mouth)
leftEAR = eye_aspect_ratio(leftEye)
rightEAR = eye_aspect_ratio(rightEye)
ear = (leftEAR + rightEAR) / 2.0
diff_ear = np.abs(leftEAR - rightEAR)

nose_point = (nose[3, 0], nose[3, 1])

# Compute the convex hull for the left and right eye, then
# visualize each of the eyes
mouthHull = cv2.convexHull(mouth)
leftEyeHull = cv2.convexHull(leftEye)

```

```

        SCROLL_MODE = not SCROLL_MODE
        # INPUT_MODE = not INPUT_MODE
        EYE_COUNTER = 0

        # nose point to draw a bounding box around it

    else:
        EYE_COUNTER = 0
        WINK_COUNTER = 0

if mar > MOUTH_AR_THRESH:
    MOUTH_COUNTER += 1

    if MOUTH_COUNTER >= MOUTH_AR_CONSECUTIVE_FRAMES:
        # if the alarm is not on, turn it on
        INPUT_MODE = not INPUT_MODE
        # SCROLL_MODE = not SCROLL_MODE
        MOUTH_COUNTER = 0
        ANCHOR_POINT = nose_point

    else:
        MOUTH_COUNTER = 0

if INPUT_MODE:
    cv2.putText(frame, "READING INPUT!", (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, RED_COLOR, 2)
    x, y = ANCHOR_POINT
    nx, ny = nose_point
    w, h = 60, 35
    multiple = 1
    cv2.rectangle(frame, (x - w, y - h), (x + w, y + h), GREEN_COLOR, 2)
    cv2.line(frame, ANCHOR_POINT, nose_point, BLUE_COLOR, 2)

    dir = direction(nose_point, ANCHOR_POINT, w, h)
    cv2.putText(frame, dir.upper(), (10, 90), cv2.FONT_HERSHEY_SIMPLEX, 0.7, RED_COLOR, 2)
    drag = 18
    if dir == 'right':

```

```
import numpy as np
```

```
# Returns EAR given eye landmarks
def eye_aspect_ratio(eye):
    # Compute the euclidean distances between the two sets of
    # vertical eye landmarks (x, y)-coordinates
    A = np.linalg.norm(eye[1] - eye[5])
    B = np.linalg.norm(eye[2] - eye[4])

    # Compute the euclidean distance between the horizontal
    # eye landmark (x, y)-coordinates
    C = np.linalg.norm(eye[0] - eye[3])

    # Compute the eye aspect ratio
    ear = (A + B) / (2.0 * C)

    # Return the eye aspect ratio
    return ear
```

```
# Returns MAR given eye landmarks
def mouth_aspect_ratio(mouth):
    # Compute the euclidean distances between the three sets
    # of vertical mouth landmarks (x, y)-coordinates
    A = np.linalg.norm(mouth[13] - mouth[19])
    B = np.linalg.norm(mouth[14] - mouth[18])
    C = np.linalg.norm(mouth[15] - mouth[17])

    # Compute the euclidean distance between the horizontal
    # mouth landmarks (x, y)-coordinates
    D = np.linalg.norm(mouth[12] - mouth[16])

    # Compute the mouth aspect ratio
    mar = (A + B + C) / (2 * D)
```

```
# Return direction given the nose and anchor points.
def direction(nose_point, anchor_point, w, h, multiple=1):
    nx, ny = nose_point
    x, y = anchor_point

    if nx > x + multiple * w:
        return 'right'
    elif nx < x - multiple * w:
        return 'left'

    if ny > y + multiple * h:
        return 'down'
    elif ny < y - multiple * h:
        return 'up'

    return '-'
```
